

Comparing Item-to-Item and Profile-Based Content Recommendations

LLM4Rec - Assignment A02

Student: Kuznetsov Vladimir

Team: Individual

Email: vpkuznetsov_1@edu.hse.ru

Date: 2026-09-22

Assignment: A02 - Content-Based Movie Recommender

Abstract

This assignment extends a baseline content-based movie recommender using the MovieLens 100K dataset and compares two recommendation modes: Item-to-Item and Profile-Based recommendation. The implementation corrects the movie genre representation, replaces Jaccard similarity with normalized cosine similarity, and generates Top-5 recommendations using either one selected movie or an averaged profile of three watched movies. The experiments show that Item-to-Item recommendations can produce a focused set of highly similar movies, while averaging movies with different genres can result in a flatter similarity distribution. The comparison also uses rating counts and manual tests to analyze recommendation behavior, discovery, and limitations of a simple content-based system.

1. Introduction

The starting point was an existing content-based recommender built on MovieLens 100K. The baseline represented movies through genres and recommended similar items using genre overlap. The goal of A02 was to compare two content-based approaches: Item-to-Item, based on one selected movie, and Profile-Based, based on an averaged profile of three watched movies.

Inspection of the baseline revealed an off-by-one feature error: genre values were extracted with `fields.slice(5, 24)`. In the supplied MovieLens `u.item` layout, field 5 is the unknown flag, so the genre mapping was shifted and Western was omitted. The implementation corrected the representation, replaced Jaccard with normalized cosine similarity, added Top-5 ranking and a three-movie profile, excluded watched items, and calculated rating counts for later analysis.

The final system was checked through read-only numerical reproduction of the implemented logic and manual browser testing. The experiments focus on correctness, score distributions, popularity/discovery, and the effect of averaging coherent versus divergent genre profiles.

2. Related Work

MovieLens 100K is a stable benchmark dataset containing 100,000 ratings and is widely used for recommender-system experiments [1, 2]. Content-based recommenders compare item features with a representation of user preference; genre vectors provide a simple and interpretable representation for this assignment [3].

Cosine similarity compares vector direction while normalizing for vector length, which is useful when movies contain different numbers of active genre features [4]. A profile can be formed by aggregating several item vectors; here, a simple element-wise mean is used so that the same cosine function can rank candidates against either a single movie or a multi-item profile.

3. Method

3.1 Feature representation

Each movie is represented by an 18-dimensional binary genre vector in the existing order from Action through Western. The baseline used `fields.slice(5, 24)`; the corrected implementation uses `fields.slice(6, 24)`, excluding the non-genre unknown field and restoring Western. Each movie keeps `{id, title, genres, vector}`. Unforgiven (1992), for example, is correctly represented as Western.

After the correction, two dataset records have no active genre features: ID 267 (unknown) and ID 1373 (Good Morning (1971)). They therefore produce valid zero vectors rather than a false genre assignment.

3.2 Cosine similarity and shared ranking

For vectors a and b , the implementation calculates $\cos(a,b) = (a \cdot b) / (||a|| \ ||b||)$. The dot product and both norms are computed explicitly. If either norm is zero, similarity is defined as 0, preventing NaN values for zero-genre records.

A shared `getTopN(queryVector, excludelds, 5)` function scores all eligible movies, sorts by cosine similarity in descending order, and returns exactly five candidates. This keeps the ranking logic identical between the two recommendation modes.

3.3 Item-to-Item and Profile-Based modes

Item-to-Item: the selected movie vector becomes the query vector; the selected ID is excluded before ranking. Profile-Based: the user selects three different watched movies and the profile is the element-wise mean $p = (v_1 + v_2 + v_3) / 3$. All three watched IDs are excluded. The same cosine function and Top-5 ranking are then applied to the resulting fractional profile vector.

3.4 Popularity measure

For analysis only, `ratingCount(itemId)` is the number of records in `u.data` for that movie. Rating count is displayed with each recommendation but is not part of the ranking score. Across the 1,682 movies parsed in this implementation, the median rating count calculated from `u.data` was 27.

3.5 Tools and verification

Tool / component	Role
OpenCode Web / AI coding agent	Baseline audit, implementation planning, minimal code changes, and read-only verification
HTML / CSS / vanilla JavaScript	Browser application and recommendation logic
MovieLens 100K: <code>u.item</code> , <code>u.data</code>	Genre metadata and rating interactions
Python 3	Read-only numerical reproduction of the implemented calculations
Chrome	Manual functional and UI testing

4. Experiments

4.1 Item-to-Item: Toy Story (1995)

Rank	Recommendation	Cosine	Ratings
1	Aladdin and the King of Thieves (1996)	1.0000	26
2	Aladdin (1992)	0.8660	219
3	Goofy Movie, A (1995)	0.8660	20
4	Santa Clause, The (1994)	0.8165	82
5	Home Alone (1990)	0.8165	137

Toy Story itself was excluded. The score range was 0.1835. The first result has the same three active genres as Toy Story and therefore receives cosine 1.0000. Manual browser testing reproduced the same Top-5, scores, genres, and rating counts.

4.2 Divergent Profile-Based recommendation

The profile combined Toy Story (1995), GoldenEye (1995), and Unforgiven (1992), intentionally mixing Animation/Children's/Comedy, Action/Adventure/Thriller, and Western preferences.

Rank	Recommendation	Cosine	Ratings
1	Muppet Treasure Island (1996)	0.6761	84
2	Space Jam (1996)	0.6761	93
3	Hercules (1997)	0.6761	66
4	Pagemaster, The (1994)	0.6761	12
5	Maverick (1994)	0.6547	128

None of the three watched IDs appeared in the result. The Top-5 score range was only 0.0215, much flatter than the Toy Story Item-to-Item run. The same list was reproduced manually in the browser.

4.3 Effect of profile averaging

To test whether the flat scores were related to combining dissimilar movies, a second profile used Star Wars (1977), Return of the Jedi (1983), and The Empire Strikes Back (1980). These movies have strongly overlapping genre vectors.

Measure	Divergent profile	Coherent profile
Watched-to-profile cosine	0.6547 / 0.6547 / 0.3780	0.9891 / 0.9891 / 0.9631
Highest Top-5 score	0.6761	0.8847
Top-5 score range	0.0215	0.1185

In these two runs, averaging the divergent vectors produced a lower and flatter recommendation distribution, while the coherent profile stayed much closer to its watched movies. This is evidence from the tested profiles, not a universal claim about all diverse user histories.

4.4 Popularity and discovery

The Item-to-Item Top-5 rating counts were 26, 219, 20, 82, 137 (median 82). The divergent Profile-Based counts were 84, 93, 66, 12, 128 (median 84). Both lists contained a movie below the catalog median of 27 ratings: Goofy Movie, A had 20 and Pagemaster, The had 12. These examples show that the tested lists were not composed exclusively of highly rated-count movies, but a larger evaluation would be required to make a general claim about popularity bias.

4.5 Manual functional testing

Verification	Observed result
Item-to-Item Top-5	PASS - Toy Story returned exactly five results matching numerical reproduction
Profile-Based Top-5	PASS - three watched movies returned the expected five results
Watched-item exclusion	PASS - IDs 1, 2, and 203 did not appear in the profile Top-5
Duplicate validation	PASS - duplicate watched selections were rejected
Genre mapping / zero vector	PASS - Unforgiven is Western; zero-genre records are handled without NaN
Popularity display	PASS - rating counts are shown with recommendations

5. Discussion

5.1 Recommendation behavior and profile averaging

The two modes produced clearly different behavior in the tested examples. Item-to-Item recommendation for Toy Story stayed concentrated around Animation, Children's, and Comedy and produced relatively high similarities. The divergent profile mixed several unrelated genre groups, and its Top-5 scores occupied a narrow 0.0215 range. The two Top-5 lists had no movies in common.

The coherent-versus-divergent comparison illustrates a limitation of simple profile averaging. When watched movies have similar genre vectors, their average remains a specific preference direction. When they are very different, weight is spread across unrelated genres and several candidates can become almost equally similar. More advanced systems could use weighted profiles, multiple preference clusters, or richer item features, but those extensions were intentionally outside this assignment.

5.2 Popularity and discovery

Rating count does not influence ranking in this implementation; cosine uses only genre vectors. The tested lists included both frequently rated and below-median items. This is useful evidence about the observed runs, but it is not sufficient to conclude that either mode systematically reduces popularity bias.

5.3 Dataset and implementation limitations

Manual inspection found a real dataset entry titled unknown. After the genre-mapping correction it has a zero genre vector, which the zero-norm cosine rule handles safely. Several non-ASCII titles also render with replacement characters in the browser, including the records corresponding to Koldum klaka (Cold Fever), JLG/JLG - autoportrait de decembre, and C'est arrive pres de chez vous. This is documented as a title-decoding/display limitation; recommendation math uses separate genre fields and is unaffected.

The implementation also leaves equal-score ordering to the existing dataset order rather than adding a separate tie-breaker. Finally, preference is represented only by genres: rating values, actors, directors, plot text, and other signals are not used.

6. AI Usage Disclosure

An AI coding agent was used to inspect the existing project, understand the baseline, identify defects, compare implementation options, and make the selected minimal changes. The agent corrected the feature representation, replaced Jaccard similarity with normalized cosine similarity, added shared Top-5 ranking and Profile-Based recommendation, excluded watched items, and added rating counts for analysis.

A separate read-only verification reproduced the implemented calculations against the MovieLens data using Python. This numerical reproduction did not modify project files. ChatGPT (GPT-5.6 Sol) was used for step-by-step workflow guidance, interpretation of results, manual test planning, and drafting this report.

Personally verified. I manually tested the final application in the browser. I checked the Toy Story Item-to-Item Top-5, the Toy Story + GoldenEye + Unforgiven profile, watched-item exclusion, duplicate-selection validation, and the visible title-encoding anomalies. AI output was not treated as verification by itself.

Audit trail. The exported OpenCode session JSON submitted with the assignment records the coding-agent interaction and tool/file activity. ChatGPT activity outside OpenCode is disclosed separately above.

References

- [1] GroupLens Research, "MovieLens 100K Dataset." [Online]. Available: <https://grouplens.org/datasets/movielens/100k/>. [Accessed: 2026-09-22].
- [2] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," ACM Transactions on Interactive Intelligent Systems, vol. 5, no. 4, 2015. [Online]. Available: <https://doi.org/10.1145/2827872>. [Accessed: 2026-09-22].
- [3] F. Ricci, L. Rokach, and B. Shapira, Eds., Recommender Systems Handbook, 3rd ed. Springer, 2022. [Online]. Available: <https://doi.org/10.1007/978-1-0716-2197-4>. [Accessed: 2026-09-22].
- [4] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008, ch. 6, "Scoring, term weighting and the vector space model." [Online]. Available: <https://nlp.stanford.edu/IR-book/html/htmledition/dot-products-1.html>. [Accessed: 2026-09-22].